

ECEGR 2220 01 26SQ Microprocessor Design : Final Project

Design and Implementation of a Single-Cycle LEGv8 Processor

Goal

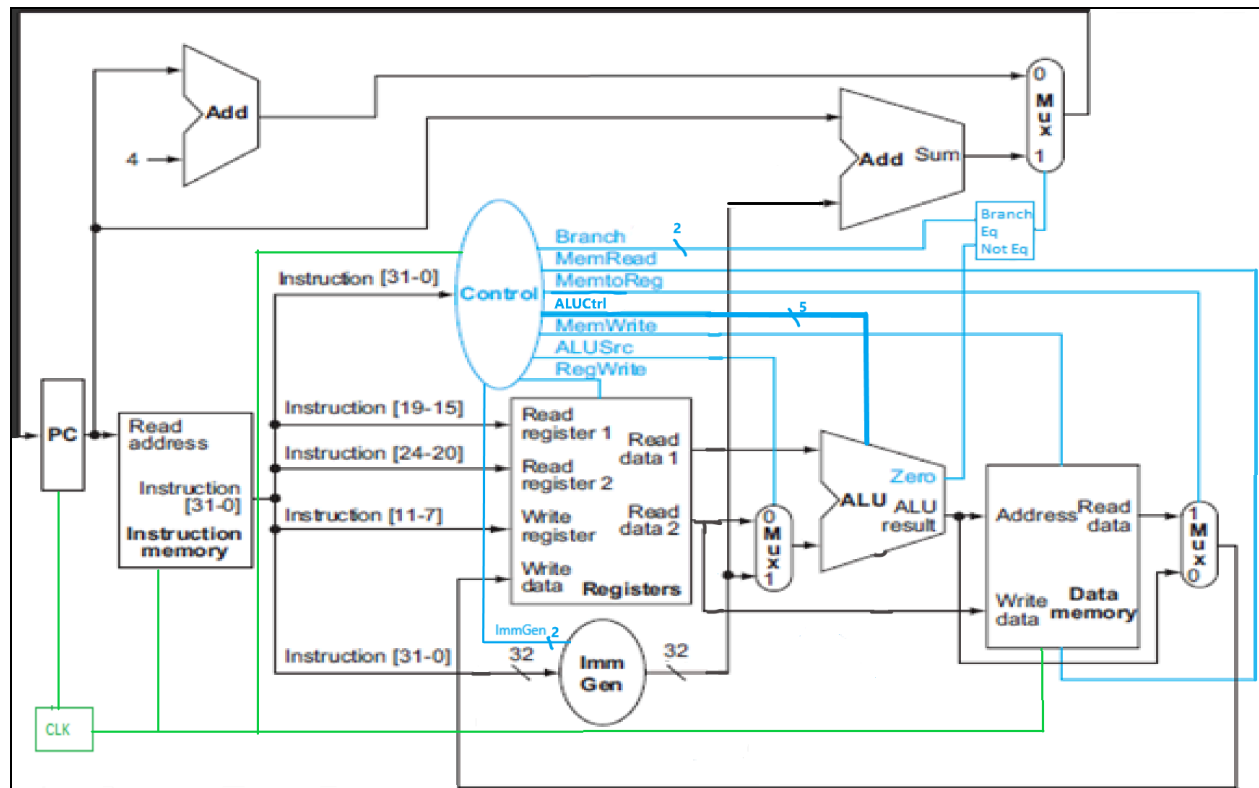
In this final project, you will design and implement a complete **single-cycle LEGv8 processor** using the hardware modules developed throughout the quarter.

You will build upon:

- Lab 2: Adders and Subtractors
- Lab 3: Registers, MUXes, and Shifters
- Lab 4: Multiplier, Divider, Floating-Point Arithmetic
- Lab 5: LEGv8 Single-Cycle/Pipeline Simulator Study
- Lab 6: Cache Controller and Address Decoder
- Lab 7: Memory and Register Bank

The goal is to integrate these modules into a working processor capable of executing a subset of LEGv8 instructions.

Students may work individually or in groups of up to three students.



Before writing any code, identify the purpose of each block in the provided datapath diagram.

#	Block	Function
1	32-bit PC	Holds address of current instruction
2	Instruction Memory	Stores program instructions
3	Register File	Stores processor registers
4	Immediate Generator	Produces sign-extended immediate values
5	ALU	Performs arithmetic and logical operations
6	Data Memory	Stores program data
7	Control Unit	Generates control signals
8	MUXes	Select data paths

Draw arrows on the datapath showing the execution flow for:

ADD X3, X1, X2 and LDUR X4, [X1,#8]

Students must demonstrate correct execution of the required instruction set:

SUB

ADD

AND

ORR

ADDI

LDUR

STUR

CBZ

B

The simulation should clearly show:

- Program Counter updates
- Register reads and writes
- ALU operations
- Memory accesses
- Branch behavior
- Correct final register and memory contents

Unique Extension Feature

Each group must implement one extension.

No two groups may choose the same extension.

Examples provided in the Extensions File.

Grading Rubric (100 Points)

The processor consists of eight major modules. Each module will be evaluated independently for correct functionality, integration, and simulation results.

#	Module	Points
1	Program Counter (PC)	10
2	Instruction Memory	10
3	Register File	10
4	Immediate Generator	5
5	ALU	10
6	Data Memory	10
7	Control Unit and Branch Logic	10
8	Multiplexers (MUXes)	7
9	Complete Processor Simulation and Verification and demo	20
10	Extension Feature	8
	Total	100

Suggested Mapping Between Labs and Final Project

Students will be provided with the following starter files:

- InstructionMemory.vhd – contains a sample LEGv8 machine-code program.
- Processor.vhd – a top-level processor skeleton showing the major datapath connections.
- tProcessor.vhd – a testbench that generates the clock and reset signals required for simulation.

The primary coding tasks involve implementing and integrating the Program Counter (PC), Register File connections, Immediate Generator, ALU, Data Memory interface, Control Unit, Multiplexers, and Branch Logic within the provided processor framework. Students are encouraged to reuse and adapt modules developed in previous labs whenever possible. After individually verifying each module, students must integrate them into a complete single-cycle LEGv8 processor capable of executing the required instruction set. Finally, each group must implement and demonstrate one approved extension feature beyond the base processor functionality.

Final Project Module	Previous Lab
PC	Lab 3
Instruction Memory	Lab 7 Memory Concepts
Register File	Lab 7
Immediate Generator	Lab 3
ALU	Labs 2, 3, and 4
Data Memory	Lab 7
Control Unit	Lab 5 Concepts
MUXes	Lab 3
Extension Feature	Labs 3, 4, 6, or 8

This project is intended to be an integration of the work completed throughout the quarter rather than a completely new design. Students are encouraged to reuse and improve their existing modules whenever appropriate.

Implementation Guidelines

The following sections describe the functionality and implementation requirements of each processor module. Students are encouraged to develop and verify each module independently before integrating them into the complete processor.

1. Program Counter (PC)

The Program Counter (PC) stores the address of the current instruction being executed.

Requirements

- Implement a 32-bit register.
- Update the PC on the rising edge of the clock.
- Reset initializes the PC to address 0.
- Under normal operation, the PC increments by 4 after each instruction.
- During branch instructions, the PC must update to the branch target address.

Verification

Demonstrate:

- PC reset to 0.
- Sequential updates (0, 4, 8, 12, ...).
- Correct branch target updates.

2. Instruction Memory

The Instruction Memory stores the machine-code program executed by the processor.

Requirements

- Implement as a ROM or initialized memory array.
- Input: PC address.
- Output: 32-bit instruction.
- A starter instruction memory file will be provided. (InstructionMemory.vhd)

Verification

Demonstrate:

- Different PC values produce the correct instructions.
- Instructions are fetched correctly during simulation.

3. Register File

The Register File stores processor registers and provides operands for ALU operations.

Requirements

- Minimum support for registers X0–X7.
- Implement the XZR register, which always outputs 0.
- Two read ports.
- One write port.
- Register writes occur when RegWrite is asserted.

Verification

Demonstrate:

- Register read operations.
- Register write operations.
- Simultaneous dual-register reads, Correct XZR behavior.

4. Immediate Generator

The Immediate Generator extracts immediate fields from instructions and generates sign-extended values.

Requirements

Support immediates for:

- ADDI
- LDUR
- STUR
- CBZ
- B

The output should be a properly sign-extended 32-bit value.

Verification

Demonstrate:

- Positive immediate values.
- Negative immediate values.
- Correct sign extension.

5. Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations.

Required Operations

ALU Operation	Description
ADD	Addition
SUB	Subtraction
AND	Bitwise AND
ORR	Bitwise OR

Inputs

- Operand A
- Operand B
- ALU Control Signal

Outputs

- ALU Result
- Zero Flag

Verification

Demonstrate each ALU operation independently before processor integration.

6. Data Memory

The Data Memory stores program data.

Requirements

Support:

- LDUR (memory read)
- STUR (memory write)

Inputs:

- Address
- Write Data
- MemRead
- MemWrite

Outputs:

- Read Data

Verification

Demonstrate:

- Successful memory writes.
- Successful memory reads.
- Correct address generation.

7. Control Unit

The Control Unit generates all control signals required for instruction execution.

Requirements

Implement the Control Unit using combinational logic.

The Control Unit must decode instruction opcodes and generate appropriate values for:

- RegWrite
- ALUSrc
- MemRead
- MemWrite
- MemtoReg
- Branch
- ALUControl

Verification

Create and verify a control table showing all generated control signals for each supported instruction.

8. Multiplexers (MUXes)

Multiplexers select the correct datapath inputs during instruction execution.

Required MUXes

ALU Source MUX

Selects between:

- Register operand
- Immediate operand

Writeback MUX

Selects between:

- ALU result
- Memory data

PC Selection MUX

Selects between:

- PC + 4
- Branch target address

Verification

Demonstrate correct selection behavior for all supported instructions.

Processor Integration

After all modules have been individually verified, integrate them into a complete single-cycle LEGv8 processor.

The processor should execute the following sequence during each clock cycle:

1. Fetch instruction from Instruction Memory.
2. Decode instruction using the Control Unit.
3. Read operands from the Register File.
4. Generate immediate value if required.
5. Perform ALU operation.
6. Access Data Memory if required.
7. Write results back to the Register File.
8. Update the Program Counter.

Recommended Development Order

Students are strongly encouraged to implement and verify modules in the following order:

1. Program Counter
2. Instruction Memory
3. Register File
4. Immediate Generator
5. ALU
6. Data Memory
7. Multiplexers
8. Control Unit
9. Complete Processor Integration

This incremental approach significantly simplifies debugging and verification.

Quartus Prime Implementation and Simulation Instructions

Students should use Quartus Prime and Questa Simulator (Intel FPGA Edition) to develop and verify their processor design.

Step 1: Create a New Quartus Project

1. Launch Quartus Prime.
2. Select **File** → **New Project Wizard**.
3. Create a new project directory.
4. Set the project name (for example, LEGv8_CPU).
5. Add all provided and student-created VHDL files to the project.

At a minimum, your project should contain:

- InstructionMemory.vhd
- Processor.vhd
- tProcessor.vhd
- PC.vhd
- RegisterFile.vhd
- ImmediateGenerator.vhd
- ALU.vhd
- DataMemory.vhd
- ControlUnit.vhd
- Any additional files required by your design

The tProcessor.vhd will be the top-level entity, followed by the Processor.vhd, which in turn utilizes the implementation from the rest of the files.

Step 2: Compile the Design

Compile the project using:

Processing → **Start Compilation**

Correct any syntax or compilation errors before proceeding.

Step 3: Run Simulation

Launch Questa Simulator through:

Tools → **Run Simulation Tool** → **RTL Simulation**

The provided testbench (tProcessor .vhd) will automatically generate:

- Clock signal
- Reset signal

and will instantiate the processor under test.

In the Questa command window, run:

```
vsim work.tProcessor  
add wave *  
run 1000 ns
```

Adjust the simulation time if necessary.

Step 4: Verify Processor Operation

The simulation should clearly demonstrate:

- Program Counter updates
- Instruction fetches
- Register reads and writes
- ALU operations
- Memory accesses
- Branch behavior
- Correct final register and memory contents

Students should capture waveform screenshots showing the successful execution of the provided test program.

FPGA Implementation

FPGA implementation is **optional** and is not required for project credit.

Students who wish to implement the processor on the DE1/DE2 FPGA board may do so, but no additional credit will be awarded.

Because this project focuses on processor architecture and integration, grading will be based entirely on simulation results and demonstration of correct processor functionality.

Deliverables

Students must submit:

1. All VHDL source files.
2. Waveform screenshots.
3. Final project report.
4. Demonstration of the processor simulation.
5. Demonstration of the extension feature.